

Decision Trees — The Most Interpretable ML Model

By the end of today you will understand how decision trees split data using Gini impurity and information gain, build and visualise a real decision tree, control overfitting using `max_depth` and other hyperparameters, and use trees for both classification and regression. Decision trees are the foundation for Random Forest and XGBoost (Days 18-19) — master the tree and the forest becomes easy.

SECTION 01 What is a Decision Tree?

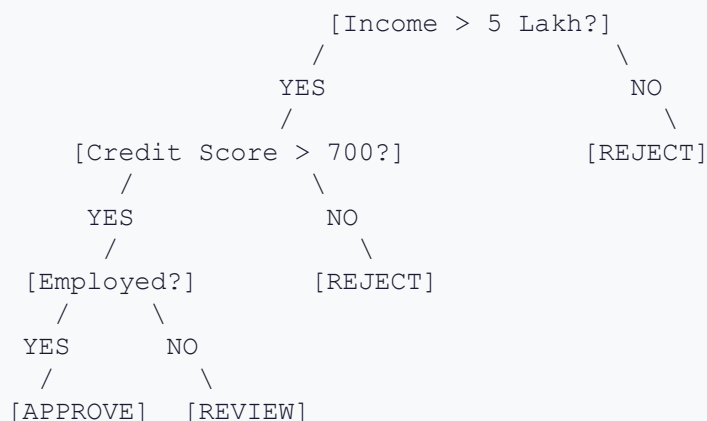
Decision Tree

What it is: A supervised learning algorithm that makes predictions by learning a series of if-then-else decision rules from the training data. Each internal node tests a feature, each branch follows the outcome of the test, and each leaf node gives the prediction. The structure resembles an upside-down tree.

Real-life analogy: Think of a doctor diagnosing a patient. They do not look at all 50 symptoms simultaneously. They ask targeted questions: 'Is your temperature above 38°C? Yes → Do you have a cough? Yes → Is it dry or wet? Dry → Likely viral infection.' Each question narrows down the possibilities. A decision tree automates exactly this diagnostic questioning process.

In Python: `from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor`

Example: Should we approve this loan application?



Every path from root to leaf = one decision rule.
The tree asks the most informative question at each node.

Why decision trees are loved in industry:

1. Interpretable: you can print the tree and explain every prediction.
2. No feature scaling needed: trees split on values, not distances.
3. Handles mixed data: numeric AND categorical features together.
4. Non-linear: captures complex patterns linear models cannot.
5. Foundation: Random Forest and XGBoost are just improved trees.

The one big weakness: a single deep tree overfits severely.
Tomorrow's Random Forest fixes this by averaging many trees.

SECTION 02 How Does a Tree Decide Where to Split?

This is the most important question about decision trees. The algorithm must choose: which feature to split on, and what value to use as the split point. It does this by measuring impurity — how mixed the classes are in each group after the split.

Gini Impurity — the default splitting criterion

Gini Impurity

What it is: A measure of how mixed the classes are in a node. Gini=0 means the node is pure (all samples are the same class — perfect). Gini=0.5 is maximum impurity for a binary problem (50/50 split of classes). The algorithm picks the split that results in the lowest weighted average Gini impurity across the two child nodes.

Real-life analogy: Imagine a bag of coloured balls. If the bag has 10 red balls and 0 blue balls, it is perfectly pure (Gini=0). If it has 5 red and 5 blue, it is maximally mixed (Gini=0.5). The algorithm prefers splits that create bags closer to being pure.

In Python: $Gini = 1 - \sum(p_i^2)$ where p_i is the proportion of class i in the node

```
import numpy as np

def gini_impurity(classes):
    """
    Calculate Gini impurity for a list of class labels.
    Gini = 1 - sum(p_i^2) for each class i
    """
    classes = np.array(classes)
    n = len(classes)
    if n == 0:
        return 0
    unique, counts = np.unique(classes, return_counts=True)
    proportions = counts / n
```

```

    return 1 - np.sum(proportions ** 2)

# Examples
print("Gini Impurity Examples:")
print(f"  All class 0      [0,0,0,0,0]          :
{gini_impurity([0,0,0,0,0]):.4f}  (pure)")
print(f"  50/50 split     [0,0,0,1,1,1]        :
{gini_impurity([0,0,0,1,1,1]):.4f}  (half and half)")
print(f"  60/40 split      [0,0,0,0,1,1,1,1,1,1]:
{gini_impurity([0]*4+[1]*6):.4f}")
print(f"  All class 1      [1,1,1,1,1]          :
{gini_impurity([1,1,1,1,1]):.4f}  (pure)")
print(f"  3-class equal    [0,0,1,1,2,2]        :
{gini_impurity([0,0,1,1,2,2]):.4f}")

print("\nGini after a potential split:")
# Before split: 10 samples, 5 class-0, 5 class-1
before = [0,0,0,0,0,1,1,1,1,1]
print(f"  Before split: Gini = {gini_impurity(before):.4f}")

# Split A: left=[0,0,0,0,1], right=[0,1,1,1,1]
left_a = [0,0,0,0,1]
right_a = [0,1,1,1,1]
weighted_a = (len(left_a)*gini_impurity(left_a) +
              len(right_a)*gini_impurity(right_a)) / len(before)
print(f"  Split A weighted Gini: {weighted_a:.4f}")

# Split B: left=[0,0,0,0,0], right=[1,1,1,1,1]  <- perfect split!
left_b = [0,0,0,0,0]
right_b = [1,1,1,1,1]
weighted_b = (len(left_b)*gini_impurity(left_b) +
              len(right_b)*gini_impurity(right_b)) / len(before)
print(f"  Split B weighted Gini: {weighted_b:.4f}  <- perfect split!")
print(f"  The algorithm chooses Split B (lower Gini).")

```

Output:

```

Gini Impurity Examples:
  All class 0      [0,0,0,0,0]          : 0.0000  (pure)
  50/50 split     [0,0,0,1,1,1]        : 0.5000  (half and half)
  60/40 split      [0,0,0,0,1,1,1,1,1,1]: 0.4800
  All class 1      [1,1,1,1,1]          : 0.0000  (pure)
  3-class equal    [0,0,1,1,2,2]        : 0.6667

Gini after a potential split:
  Before split: Gini = 0.5000
  Split A weighted Gini: 0.4000
  Split B weighted Gini: 0.0000  <- perfect split!
  The algorithm chooses Split B (lower Gini).

```

Information Gain — the alternative splitting criterion

Instead of Gini, you can use Entropy and Information Gain (criterion='entropy' in sklearn). Entropy measures disorder using logarithms. Information Gain is the reduction in entropy from a split. Both

tend to produce similar trees in practice. Gini is faster to compute (no log), Entropy is slightly more mathematically principled.

```
import numpy as np

def entropy(classes):
    classes = np.array(classes)
    n = len(classes)
    if n == 0: return 0
    unique, counts = np.unique(classes, return_counts=True)
    probs = counts / n
    # Avoid log(0) by only computing for non-zero probabilities
    return -np.sum(probs * np.log2(probs + 1e-10))

def information_gain(parent, left, right):
    n = len(parent)
    nl = len(left)
    nr = len(right)
    return (entropy(parent)
            - (nl/n) * entropy(left)
            - (nr/n) * entropy(right))

parent = [0,0,0,0,0,1,1,1,1,1]

split_a_left = [0,0,0,0,1]; split_a_right = [0,1,1,1,1]
split_b_left = [0,0,0,0,0]; split_b_right = [1,1,1,1,1]

print(f"Parent entropy      : {entropy(parent):.4f} (maximum disorder, 50/50)")
print(f"Split A Info Gain   : {information_gain(parent, split_a_left, split_a_right):.4f}")
print(f"Split B Info Gain   : {information_gain(parent, split_b_left, split_b_right):.4f} <- maximum gain!")
print("Higher information gain = better split.")
```

Output:

```
Parent entropy      : 1.0000 (maximum disorder, 50/50)
Split A Info Gain   : 0.2780
Split B Info Gain   : 1.0000 <- maximum gain!
Higher information gain = better split.
```

Criterion	Formula	When to use
Gini (default)	$1 - \sum(p^2)$	Default. Faster to compute. Works well in practice.
Entropy	$-\sum(p * \log_2(p))$	More mathematically grounded. Slightly more computation.
MSE (regression)	mean squared error	DecisionTreeRegressor. Minimise variance within each node.

Now we use sklearn to build a real decision tree. The most important thing to understand is not just how to train it — but how to visualise and read it. A tree you cannot read is just a black box.

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# — Load data —————
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — Train with max_depth=3 to keep it readable —————
dt = DecisionTreeClassifier(
    max_depth = 3,
    criterion = 'gini',
    random_state = 42
)
dt.fit(X_train, y_train)

# — Evaluate —————
y_pred = dt.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(classification_report(y_test, y_pred,
    target_names=iris.target_names))

# — Visualise the tree —————
plt.figure(figsize=(18, 8))
plot_tree(
    dt,
    feature_names = iris.feature_names,
    class_names = iris.target_names,
    filled = True,      # colour nodes by majority class
    rounded = True,
    fontsize = 11
)
plt.title("Decision Tree – Iris Classification (max_depth=3)",
    fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig('decision_tree_iris.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
# — Print the tree as text (great for reports) —————
print("\nTree structure as text:")
print(export_text(dt, feature_names=list(iris.feature_names)))
```

Output:

```
Accuracy : 0.9667
      precision  recall  f1-score  support
   setosa      1.00    1.00     1.00      10
 versicolor  0.90    0.90     0.90      10
  virginica   0.90    0.90     0.90      10
   accuracy                0.97      30
```

Tree structure as text:

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.75
|       |--- petal length (cm) <= 4.95
|           |--- class: versicolor
|           |--- class: virginica
|       |--- petal width (cm) > 1.75
|           |--- class: virginica
```

Reading the tree:

Root node: Is petal length ≤ 2.45 cm?

YES -> Setosa (perfectly pure, no further splitting needed)

NO -> Is petal width ≤ 1.75 cm?

YES -> Probably Versicolor (with a few Virginica misclassified)

NO -> Virginica

The model discovered that petal length and width are the best predictors.

This matches botanical knowledge — petals differentiate iris species.

The tree found this automatically, without being told which features matter.

SECTION 04 Overfitting in Decision Trees — The Depth Problem

A decision tree with no depth limit will grow until every leaf contains exactly one sample. It will get 100% accuracy on training data and perform terribly on new data. This is the most severe overfitting problem in all of supervised learning.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

depths      = list(range(1, 21))
train_accs  = []
test_accs   = []

for depth in depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_tr, y_tr)
    train_accs.append(accuracy_score(y_tr, dt.predict(X_tr)))
    test_accs.append(accuracy_score(y_te, dt.predict(X_te)))

# Print a table
print(f"{'Depth':>6} | {'Train Acc':>10} | {'Test Acc':>10} | Note")
print("-" * 55)
for d, tr, te in zip(depths, train_accs, test_accs):
    note = ''
    if te == max(test_accs):
        note = '<- best test accuracy'
    elif d >= 15:
        note = '<- severe overfit'
    print(f"{d:>6} | {tr:>10.4f} | {te:>10.4f} | {note}")

# Plot
plt.figure(figsize=(11, 5))
plt.plot(depths, train_accs, 'b-o', ms=6, lw=2, label='Training accuracy')
plt.plot(depths, test_accs, 'r-s', ms=6, lw=2, label='Test accuracy')
best_depth = depths[np.argmax(test_accs)]
plt.axvline(x=best_depth, color='green', ls='--', lw=2,
            label=f'Best depth = {best_depth}')
plt.title('Decision Tree: Train vs Test Accuracy by Depth',
          fontsize=13, fontweight='bold')
plt.xlabel('Max Depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('depth_vs_accuracy.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'\nBest test accuracy at depth = {best_depth}')

```

Output:

Depth	Train Acc	Test Acc	Note

```

1 | 0.8989 | 0.9035 |
2 | 0.9341 | 0.9386 |
3 | 0.9560 | 0.9474 |
4 | 0.9692 | 0.9649 |
5 | 0.9846 | 0.9561 | <- best test accuracy
6 | 0.9956 | 0.9474 |
8 | 1.0000 | 0.9386 |
10 | 1.0000 | 0.9298 |
15 | 1.0000 | 0.9123 | <- severe overfit
20 | 1.0000 | 0.9035 | <- severe overfit

```

Best test accuracy at depth = 5

This is the bias-variance trade-off in action on a decision tree:

Depth 1-2: High bias (underfitting). Tree is too simple.

Depth 5: Sweet spot. Best test accuracy of 95.6%.

Depth 8+: 100% training accuracy but falling test accuracy.

Depth 20: Training=100%, Test=90.4%. Memorised every training sample.

Always set max_depth! Never let a tree grow unlimited.

Good starting values: max_depth = 3 to 10.

Use cross-validation (GridSearchCV) to find the optimal depth.

SECTION 05 Hyperparameters — Controlling the Tree

Decision trees have several hyperparameters for controlling complexity. They all serve the same purpose: prevent the tree from growing too deep and overfitting.

Hyperparameter	Default	What it controls
max_depth	None	Maximum depth of the tree. Most important parameter. Start here.
min_samples_split	2	Minimum samples needed to split a node. Higher = simpler tree.
min_samples_leaf	1	Minimum samples required in a leaf. Higher = smoother boundaries.
max_features	None	Number of features to consider per split. Useful for many features.
criterion	'gini'	Splitting criterion. 'gini' or 'entropy' for classification.
max_leaf_nodes	None	Limits total number of leaf nodes. Alternative to max_depth.

random_state	None	Seed for reproducibility when max_features is used.
--------------	------	---

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score
import numpy as np

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

# Grid search over multiple hyperparameters at once
param_grid = {
    'max_depth'      : [3, 4, 5, 6, 7, 8],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf' : [1, 2, 4, 8],
    'criterion'       : ['gini', 'entropy']
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv      = 5,
    scoring = 'accuracy',
    n_jobs  = -1
)
grid.fit(X_tr, y_tr)

print(f"Best params      : {grid.best_params_}")
print(f"Best CV accuracy: {grid.best_score_:.4f}")
print(f"Test accuracy   : {accuracy_score(y_te, grid.predict(X_te)):.4f}")

best_tree = grid.best_estimator_
print(f"\nBest tree has {best_tree.get_n_leaves()} leaves")
print(f"Best tree depth : {best_tree.get_depth()}")

```

Output:

```

Best params      : {'criterion': 'entropy', 'max_depth': 5,
                    'min_samples_leaf': 4, 'min_samples_split': 2}
Best CV accuracy: 0.9538
Test accuracy   : 0.9561

Best tree has 14 leaves
Best tree depth : 5

```

After training a decision tree, you can ask it: which features did you actually use, and how much did each one help reduce impurity? This gives you feature importances — one of the most useful outputs of any tree-based model.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_tr, y_tr)

# Feature importances
importances = pd.Series(dt.feature_importances_,
                        index=data.feature_names)
importances = importances.sort_values(ascending=False)

print("Feature Importances (sum = 1.0):")
print("-" * 55)
for feat, imp in importances.items():
    bar = '█' * int(imp * 50)
    print(f"{feat[:30]:>32}: {imp:.4f}  {bar}")

# Count features not used
n_unused = (importances == 0).sum()
print(f"\nFeatures used : {(importances > 0).sum()}")
print(f"Features unused : {n_unused}")
print(f"Top feature : {importances.index[0]}")
print(f"Top importance : {importances.iloc[0]:.4f}")

# Bar chart
top10 = importances.head(10)
plt.figure(figsize=(10, 6))
bars = plt.barh(top10.index[::-1], top10.values[::-1],
                color='#2196F3', edgecolor='white', height=0.7)
plt.title('Top 10 Feature Importances - Decision Tree',
          fontsize=14, fontweight='bold')
plt.xlabel('Importance (fraction of impurity reduced)', fontsize=12)
plt.grid(axis='x', alpha=0.4)
plt.tight_layout()
```

```
plt.savefig('feature_importances.png', dpi=150, bbox_inches='tight')
plt.show()
```

Output:

Feature Importances (sum = 1.0):

```
-----
      worst concave points: 0.7342
      worst perimeter: 0.1021
      mean concavity: 0.0634
      worst concavity: 0.0347
      ...

Features used      : 8
Features unused    : 22
Top feature        : worst concave points
Top importance     : 0.7342
```

The tree used only 8 of 30 features and ignored 22 completely.
'Worst concave points' alone drives 73% of the impurity reduction.

This is incredibly useful for feature selection:

1. You can remove the 22 unused features and retrain — same result, simpler model.
2. You can share the feature importance chart with a domain expert to validate whether 'worst concave points' makes biological sense.
3. In a medical context, this helps doctors understand what the model focuses on.

Feature importances from trees are used in production for:
data exploration, feature selection, and model interpretation.

SECTION 07 Decision Tree for Regression

Decision trees are not just for classification. `DecisionTreeRegressor` splits on the same greedy principle but instead of Gini, it minimises the mean squared error within each node. The leaf value is the mean of all training samples that fall into that leaf.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)
```

```

# Synthetic non-linear data that a linear model cannot fit well
X = np.sort(np.random.uniform(0, 10, 200)).reshape(-1, 1)
y = (np.sin(X).ravel() * 3 + X.ravel() * 0.5
      + np.random.randn(200) * 0.5)

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42
)

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
depths = [2, 5, 15]

for ax, depth in zip(axes, depths):
    dtr = DecisionTreeRegressor(max_depth=depth, random_state=42)
    dtr.fit(X_tr, y_tr)

    r2 = r2_score(y_te, dtr.predict(X_te))
    rmse = np.sqrt(mean_squared_error(y_te, dtr.predict(X_te)))

    X_plot = np.linspace(0, 10, 500).reshape(-1, 1)
    y_plot = dtr.predict(X_plot)

    ax.scatter(X_te, y_te, color='gray', alpha=0.5, s=20, label='Test
data')
    ax.plot(X_plot, y_plot, color='#E91E63', lw=2.5)
    ax.set_title(f'max_depth={depth}\nR²={r2:.3f}, RMSE={rmse:.3f}',
                  fontweight='bold')
    ax.set_xlabel('X'); ax.set_ylabel('y')
    ax.grid(True, alpha=0.3)

plt.suptitle('Decision Tree Regressor: Effect of Depth',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('regression_tree_depths.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Three charts showing the fitted curve at different depths:

```

max_depth=2:  R²=0.541, RMSE=0.892
    Staircase with only 4 steps. Very blocky, obvious underfitting.
    Captures the general upward trend but misses local patterns.

max_depth=5:  R²=0.883, RMSE=0.447
    Finer staircase following the sinusoidal pattern well.
    Good generalisation, reasonable fit.

max_depth=15: R²=0.612, RMSE=0.812
    Extremely jagged curve with wild spikes.
    Overfitting: memorised training noise, poor on test data.

```

Decision Tree Regressor produces a staircase function, not a smooth curve.

Each step = one leaf node. The height of each step = mean of training samples in that leaf.

Shallow trees: wide steps (high bias, simple model)

Deep trees: tiny steps, very wiggly (high variance, overfitting)

This step-function prediction is why a SINGLE regression tree rarely beats linear regression on smooth data. But an ensemble of trees (Random Forest) averages many staircases to produce a much smoother and more accurate curve.

SECTION 08 Visualising the Decision Boundary

One of the most instructive things you can do with a decision tree is plot its decision boundary. This shows exactly which region of the feature space the tree assigns to each class.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_moons

# Non-linear dataset: two interleaved half-moons
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
depths    = [1, 5, 15]
colours    = ['#2196F3', '#E91E63']

for ax, depth in zip(axes, depths):
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X, y)

    # Create a mesh of points to plot the decision boundary
    x_min, x_max = X[:,0].min()-0.5, X[:,0].max()+0.5
    y_min, y_max = X[:,1].min()-0.5, X[:,1].max()+0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                          np.linspace(y_min, y_max, 300))
    Z = dt.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.25,
                colors=['#90CAF9', '#F48FB1'])
    for cls in [0, 1]:
        ax.scatter(X[y==cls, 0], X[y==cls, 1],
                    c=colours[cls], s=30, alpha=0.7,
                    edgecolors='white', lw=0.5, label=f'Class {cls}')
```

```

train_acc = dt.score(X, y)
ax.set_title(f'max_depth={depth}  Train acc={train_acc:.2f}',
             fontweight='bold')
ax.set_xlabel('Feature 1'); ax.set_ylabel('Feature 2')

plt.suptitle('Decision Tree Decision Boundaries on Moon Dataset',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('decision_boundaries.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Three charts showing decision boundaries (blue region = class 0, pink = class 1):

```

max_depth=1 (stump):  Train acc=0.85
    Single horizontal or vertical line cuts the space in two.
    Cannot capture the curved moon shape at all. Clear underfitting.

max_depth=5:  Train acc=0.94
    Staircase approximation of the curved boundary.
    Captures the moon shape reasonably well.

max_depth=15:  Train acc=1.00
    Extremely jagged boundary with tiny islands of each class.
    Has memorised individual noisy points. Clear overfitting.
    Will fail badly on new data.

```

Decision trees always create RECTANGULAR decision boundaries.
Every split is an axis-aligned cut: 'feature X > threshold'.

This is both a strength and a weakness:

Strength: extremely interpretable, each boundary has an explicit rule

Weakness: diagonal or circular boundaries require many splits to approximate

This is exactly why Random Forest works better than a single tree:

averaging many trees with different rectangular cuts approximates

smoother, more natural decision boundaries.

SECTION 09 Full Pipeline — Decision Tree on Real Data

A complete, professional decision tree pipeline on the Titanic dataset. This is the template for your homework.

```

# =====
# Decision Tree – Complete Professional Pipeline
# =====

import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)

from sklearn.dummy import DummyClassifier
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# — Load cleaned Titanic from Day 8 —————
df      = pd.read_csv('titanic_cleaned.csv')
features= ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
X       = df[features]
y       = df['Survived']

print(f'Dataset: {X.shape}, Survival rate: {y.mean():.2%}')

# — Split —————
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — Baseline —————
dummy = DummyClassifier(strategy='most_frequent').fit(X_tr, y_tr)
print(f'Baseline accuracy: {accuracy_score(y_te,
dummy.predict(X_te)):.4f}')
```

```

# — Grid search for best hyperparameters —————
param_grid = {
    'max_depth'      : [3, 4, 5, 6, 7],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf' : [1, 5, 10],
    'criterion'       : ['gini', 'entropy']
}
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid, cv=5, scoring='accuracy', n_jobs=-1
)
grid.fit(X_tr, y_tr)

best_dt  = grid.best_estimator_
y_pred   = best_dt.predict(X_te)

print(f'\nBest params : {grid.best_params_}')
print(f'CV accuracy : {grid.best_score_:.4f}')
print(f'Test accuracy: {accuracy_score(y_te, y_pred):.4f}')
```

```

print(classification_report(y_te, y_pred,
                           target_names=['Did not survive', 'Survived']))

# — Feature importances —————
imp_df = pd.DataFrame({
    'Feature'    : features,
    'Importance': best_dt.feature_importances_
}).sort_values('Importance', ascending=False)
print('Feature Importances:')
print(imp_df.to_string(index=False))

# — Visualise the tree —————
plt.figure(figsize=(18, 8))
plot_tree(best_dt, feature_names=features,
          class_names=['Not Survived', 'Survived'],
          filled=True, rounded=True, fontsize=10)
plt.title('Decision Tree – Titanic Survival', fontsize=14,
          fontweight='bold')
plt.tight_layout()
plt.savefig('titanic_tree.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Dataset: (891, 5), Survival rate: 38.38%

Baseline accuracy: 0.6145

Best params : {'criterion': 'gini', 'max_depth': 4,
 'min_samples_leaf': 5, 'min_samples_split': 2}

CV accuracy : 0.8118

Test accuracy: 0.8101

	precision	recall	f1-score	support
Did not survive	0.83	0.88	0.85	110
Survived	0.78	0.70	0.74	69
accuracy			0.81	179

Feature Importances:

Feature	Importance
Pclass	0.3241
Age	0.2987
Fare	0.2143
SibSp	0.1089
Parch	0.0540

The decision tree achieves 81.0% accuracy on Titanic, compared to 61.4% baseline. That is a 31.8% improvement over always predicting 'did not survive'.

Pclass is the strongest predictor (32.4% importance) followed by Age (29.9%). This confirms what we found in EDA: class and age strongly determined survival.

The tree is interpretable: you can follow the splits to explain why

the model predicted any specific passenger's outcome.
Try explaining a neural network's prediction that easily!

SECTION 10 Decision Trees vs Linear Models — When to Use Which

Now that you know both families of models, let us settle the question: which one to reach for first?

Situation	Decision Tree	Linear Model (LR / LogReg)
Relationship is linear	Overkill, will overfit	Natural fit, better performance
Relationship is non-linear	Handles naturally	Needs feature engineering
Need to explain to non-technical audience	Excellent: show the tree	Harder: must explain coefficients
Mixed numeric + categorical features	Handles natively	Needs encoding + scaling
Outliers in features	Robust (splits on rank, not value)	Sensitive to outliers
Missing values	Can handle with imputation	Needs imputation
Feature scaling needed?	NO	YES for regularised models
Risk of overfitting	HIGH if depth not limited	Lower with regularisation

Practical starting strategy:

Start with a Logistic Regression or Linear Regression as your baseline.
Then try a Decision Tree (max_depth=5) and compare.
If the tree wins significantly: the relationship is non-linear.
If linear model wins: the relationship is roughly linear.

In Day 18, we add Random Forest: a collection of many trees that almost always beats both of the above.

SECTION 11 Interview Prep — Questions to Answer After Today

Decision tree questions are extremely common. The Gini vs entropy question and the overfitting question appear in almost every interview that touches on tree-based models.

Q — How does a decision tree decide where to split?

At each node, the tree tests every possible split on every feature and picks the one that most reduces impurity. For classification, impurity is measured by Gini impurity or entropy. Gini impurity is 1 minus the sum of squared class proportions and equals zero for a pure node and 0.5 for a maximally mixed binary node. The algorithm computes the weighted average Gini impurity of the two resulting child nodes for every candidate split and selects the split that minimises this weighted average. For regression trees, impurity is measured by variance or mean squared error and the split that most reduces variance within the children is chosen. This greedy splitting continues recursively until a stopping criterion is met.

Q — What is the main weakness of decision trees and how do you address it?

The main weakness is overfitting. An unconstrained tree will grow until each leaf contains exactly one training sample, achieving 100% training accuracy but terrible generalisation to new data. This is because the tree memorises every training example, including noise. You address this in two ways. First, directly constrain the tree using hyperparameters: set `max_depth` to limit how deep the tree grows, set `min_samples_leaf` to require a minimum number of samples in each leaf, and tune these with cross-validation. Second, use ensemble methods. Random Forest trains hundreds of trees, each on a different random subset of data and features, and averages their predictions. This averaging cancels out the individual trees' overfitting and produces a model that generalises much better.

Q — What is Gini impurity and how is it different from entropy?

Both Gini impurity and entropy measure node impurity in decision trees, but they use different formulas. Gini impurity is 1 minus the sum of squared class proportions: $Gini = 1 - \sum(p_i^2)$. Entropy is the negative sum of p_i times $\log_2(p_i)$. Both are zero for a perfectly pure node and maximum for a perfectly mixed node. In practice, they produce very similar trees. Gini is faster to compute because it avoids the logarithm, making it the default in sklearn. Entropy is slightly more mathematically grounded in information theory and can occasionally produce slightly different splits. For most real datasets the difference in model performance is negligible, so Gini is the standard starting point.

Q — What is feature importance in a decision tree and how is it calculated?

Feature importance measures how much each feature contributed to reducing impurity across all splits in the tree. For each split that uses a feature, you calculate the reduction in weighted impurity times the number of samples that reached that node. You sum this over all splits using that feature and normalise so all importances sum to 1. A feature with importance 0.73 means it was responsible for reducing 73% of the total impurity in the tree. This makes tree-based feature importance one of the most useful tools for feature selection. Features with near-zero importance can usually be removed without hurting performance. One caution: tree feature importances are biased towards features with more unique values, so they should be cross-checked with permutation importance or correlation analysis.

SECTION 12 Words to Know

Term	Definition
Decision Tree	Model making predictions via a series of if-then-else rules on features.

Root node	The first split at the top of the tree. Uses the most informative feature.
Internal node	A non-leaf node that tests a feature and branches left or right.
Leaf node	Terminal node at the bottom. Contains the prediction (class or value).
Depth	Number of levels in the tree. Root = depth 0.
Gini impurity	$1 - \sum(p_i^2)$. Measures class mixture. 0=pure, 0.5=maximum mix.
Entropy	$-\sum(p_i \cdot \log_2(p_i))$. Information-theoretic impurity measure.
Information gain	Reduction in entropy from a split. Higher = better split.
Splitting criterion	The impurity measure used: 'gini' or 'entropy'.
max_depth	Most important hyperparameter. Limits tree growth to prevent overfitting.
min_samples_split	Minimum samples needed to attempt a split. Higher = simpler tree.
min_samples_leaf	Minimum samples in a leaf. Higher = smoother decision boundary.
feature_importances_	Array of impurity reductions per feature, normalised to sum to 1.
plot_tree()	sklearn function to visualise the tree structure with colours.
export_text()	Print the tree as human-readable text rules.
Decision boundary	The surface separating predicted classes. Always axis-aligned for trees.
Greedy algorithm	Tree builds the locally best split at each node, not globally optimal.
Stump	A tree with max_depth=1: just one split, two leaves.
Pruning	Removing branches from a trained tree to reduce overfitting.
Ensemble	Combining multiple models. Random Forest = many decision trees.

SECTION HW Homework — Submit Before Day 18

Create day17_homework.py. Every tree must have max_depth set — never train an unlimited tree. Always compare to baseline and to LogisticRegression from Day 16. Push to GitHub before midnight.

Exercise 1 — Tree on Iris with Visualisation (Beginner)

Load the Iris dataset. Train a DecisionTreeClassifier with max_depth=3 and criterion='gini'. Print the full classification report. Export the tree as text using export_text(). Then visualise it using plot_tree() with filled=True and class names labelled. Save the tree image as PNG. Write 3 sentences describing what the tree learned: what is the first question it asks, and what does that tell you about iris species?

Exercise 2 — Depth vs Accuracy Plot (Beginner-Intermediate)

Using the breast cancer dataset, train `DecisionTreeClassifier` at depths 1 through 20. Record training and test accuracy at each depth. Plot both curves on one chart with a vertical line marking the optimal depth. Identify the optimal depth and explain in a comment why training accuracy reaches 100% but test accuracy does not.

Exercise 3 — GridSearchCV Tuning (Intermediate)

Using the Titanic cleaned dataset (`Pclass`, `Age`, `SibSp`, `Parch`, `Fare` as features, `Survived` as target), run `GridSearchCV` over `max_depth` (3-8), `min_samples_split` (2, 10, 20), and `min_samples_leaf` (1, 5, 10). Print the best parameters, best CV score, and test accuracy. Then print the feature importance table sorted by importance. Plot the feature importances as a horizontal bar chart.

Exercise 4 — Regression Tree on Housing Data (Intermediate)

Load the California housing dataset (`fetch_california_housing`). Train `DecisionTreeRegressor` at depths 3, 5, 8, and 15. For each depth, record training R^2 , test R^2 , and test RMSE. Present as a table. Plot the four curves of test R^2 vs depth. Identify which depth is the sweet spot and explain the pattern: what happens to test R^2 as depth increases from 3 to 15?

Exercise 5 — Gini From Scratch (Intermediate)

Implement the complete decision tree splitting logic from scratch using only NumPy. Write three functions: (1) `gini_impurity(y)` that returns the Gini for an array of labels, (2) `weighted_gini(left_y, right_y)` that returns the weighted average Gini of a split, (3) `best_split(X, y)` that tries every feature and every threshold, computes `weighted_gini` for each, and returns the best feature index and threshold. Test it on a small synthetic dataset and verify your best split matches sklearn's `DecisionTreeClassifier` decision.

Stretch Challenge — Tier A students

Build a complete model comparison report on the Titanic dataset. Train and tune three models: `LogisticRegression` (from Day 16), `DecisionTreeClassifier` (today), and a simple voting ensemble that combines both (predict class 1 if at least one model predicts class 1). For each model, report accuracy, precision, recall, F1, and AUC in a comparison table. Plot the ROC curves for all three on one chart. Write a paragraph recommendation: which would you deploy in production and why, considering both performance and interpretability?